Deliverable 2: Core Functionality Implementation

Prateek Sahay and Venkata Sai Makani

University of the Cumberlands

MSCS632-M30: Advanced Programming Languages

Jay Thom

October 3, 2026

Deliverable 2: Core Functionality Implementation

Implemented Functionality

The Expense Tracker now has independent Python and C++17 command-line implementations. Each supports add, list, date/category filters, case-insensitive text search, category totals, and an overall total. A delete-by-ID command is included as an extension. Each record stores a unique ID, date, integer cents, category, and description in the shared CSV format. The repository is https://github.com/psahay101/cross-language-expense-tracker.

Both versions validate calendar dates, positive decimal amounts, text limits, unique IDs, and existing CSV contents. Invalid input is reported without rewriting the existing data. Currency calculations use integer cents, and standard quoting preserves commas and double quotes. Saving uses a temporary file followed by replacement. The application supports one writer per data file; it intentionally does not add concurrent writes or background threads.

Language Features in the Working Code

Python's tracker.py stores expenses in dictionaries documented with TypedDict hints. It uses datetime.date, the csv module, list comprehensions for filtering, and dictionary accumulation for category totals. main.py uses argparse and catches TrackerError at the command-line boundary.

C++ defines an Expense struct, a Tracker class, std::vector record storage, std::optional filters, and std::map category totals. Fixed-width integer fields make numeric bounds explicit. Scoped file streams and a temporary-file cleanup object demonstrate resource acquisition is initialization (RAII), allowing resources to be released on exceptions without manual owning pointers. These implementations give the final comparison concrete examples of dynamic versus static typing, library support, and resource ownership.

Build, Run, and Verification

From the repository root, run Python with: python python/main.py --data data/expenses.csv add --date 2026-10-02 --amount 12.34 --category Food --description Lunch. Python 3.10 or later is required. No third-party Python packages are needed.

For the C++17 build, run cmake -S cpp -B build, followed by cmake --build build --config Release. Run ./build/expense_tracker summary on Linux/macOS, or .\build\expense_tracker.exe summary on Windows with a single-configuration build. A Windows multi-configuration generator may place the executable in build\Release instead; the README documents the applicable path.

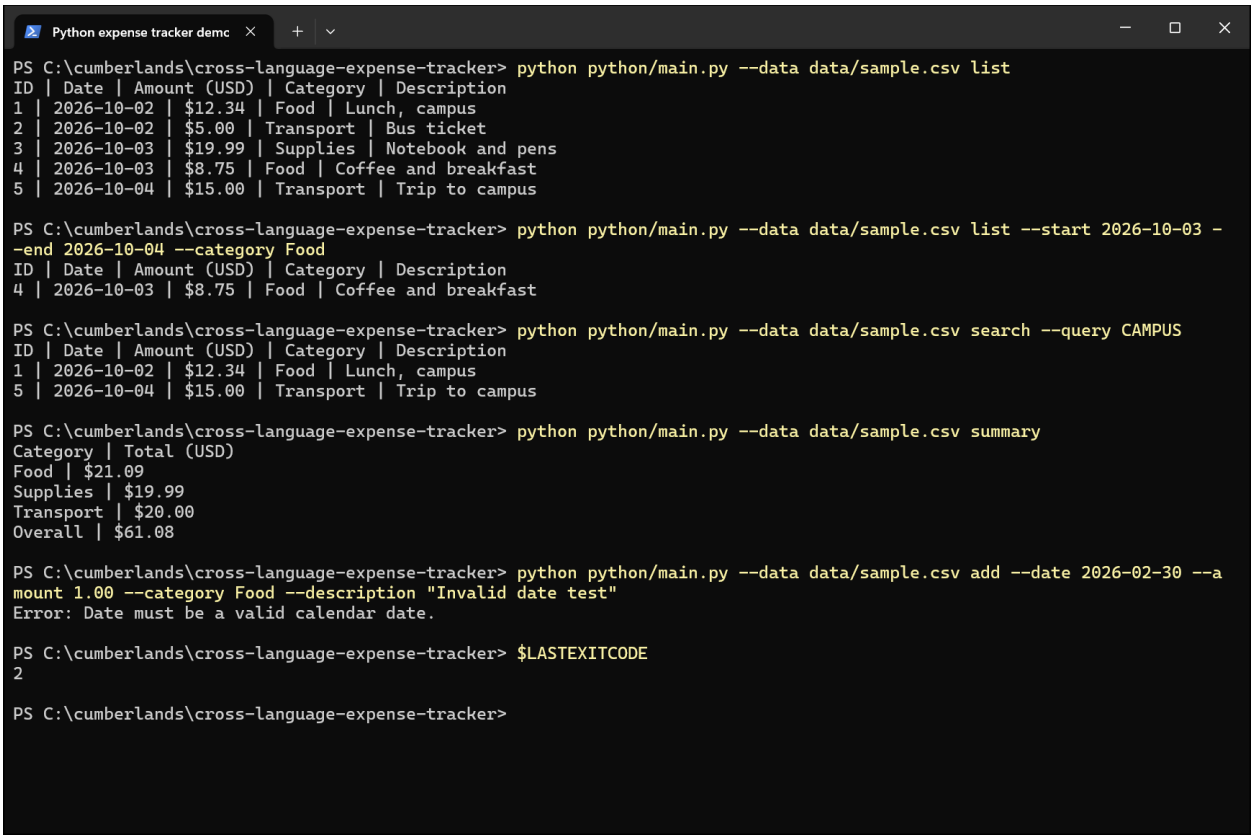
Run the shared command-line suite with python -m unittest discover -s tests -v. Set CPP_EXECUTABLE to the compiled binary if its path differs from the default build location. Run the Python core suite with python -m unittest discover -s python -p test_tracker.py -v. All 62 tests pass (55 cross-language tests plus 7 Python unit tests). The recorded output is included in docs/evidence/test_results.txt.

The tests cover exact decimal totals, date boundaries, combined filters, quoting, persistence, invalid input, corrupt-file preservation, and CSV exchange in both directions. The following figures capture command output for the demo and verification run.

Demonstration Evidence

Figure 1

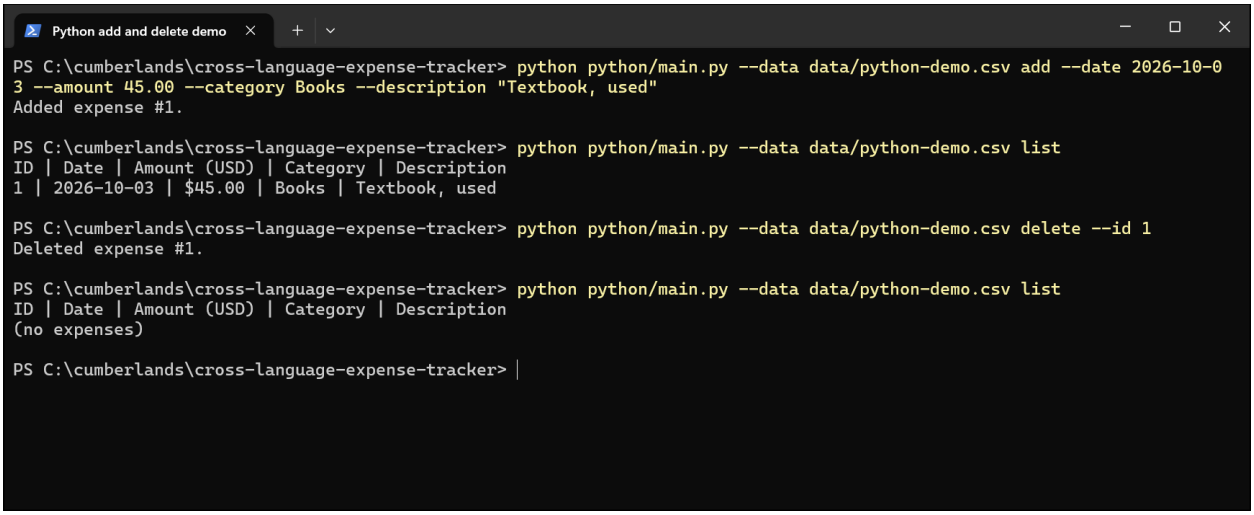
Python CLI demonstration showing recorded expenses, filtering/search, and category totals.



Note. Screenshot of recorded output from command executions. Full transcripts are included in docs/evidence.

Figure 2

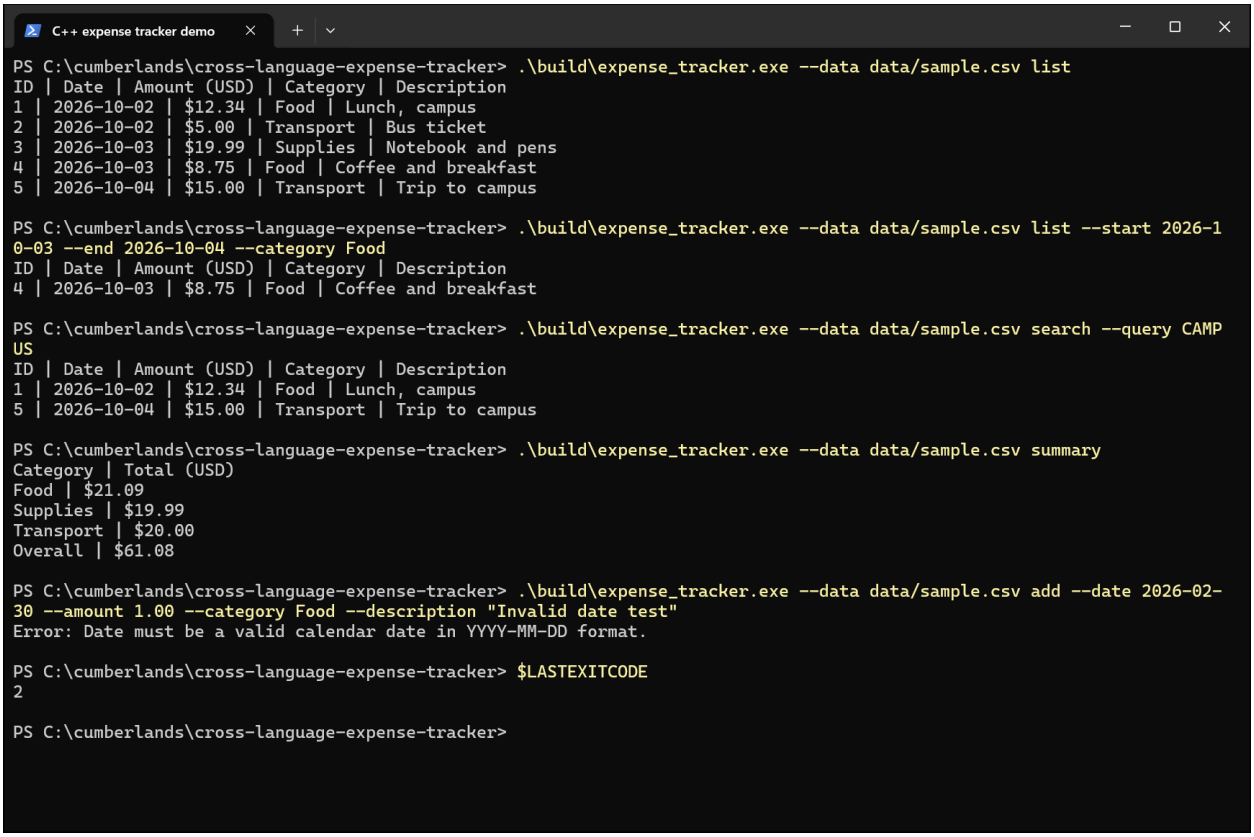
Python CLI demonstration of recording and deleting an expense.



Note. Screenshot of recorded output from command executions. Full transcripts are included in docs/evidence.

Figure 3

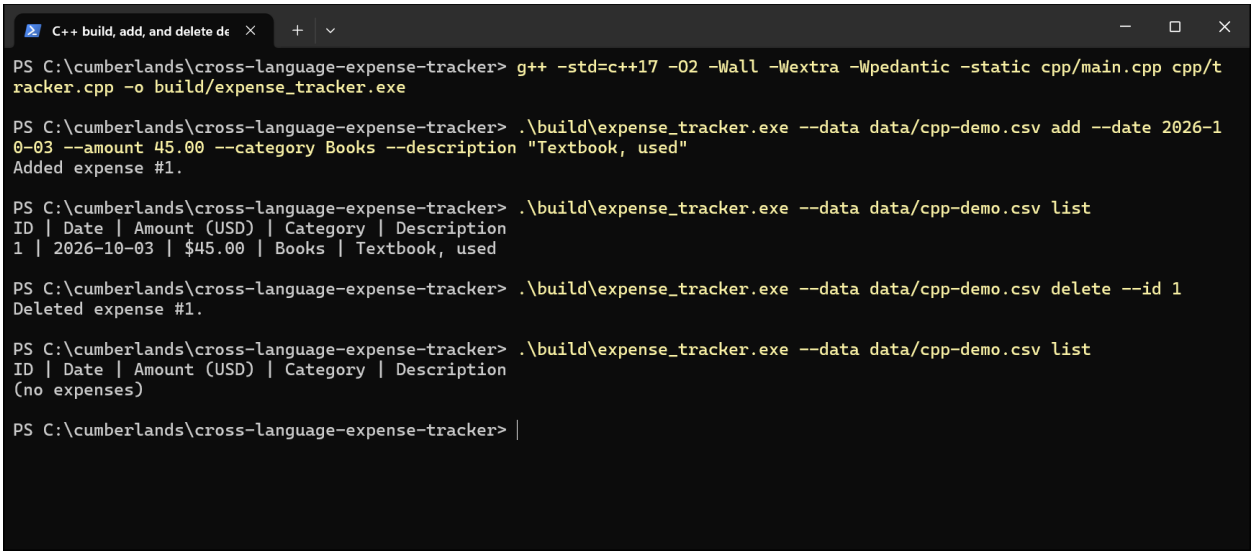
C++ CLI demonstration of the same core operations and shared output format.



Note. Screenshot of recorded output from command executions. Full transcripts are included in docs/evidence.

Figure 4

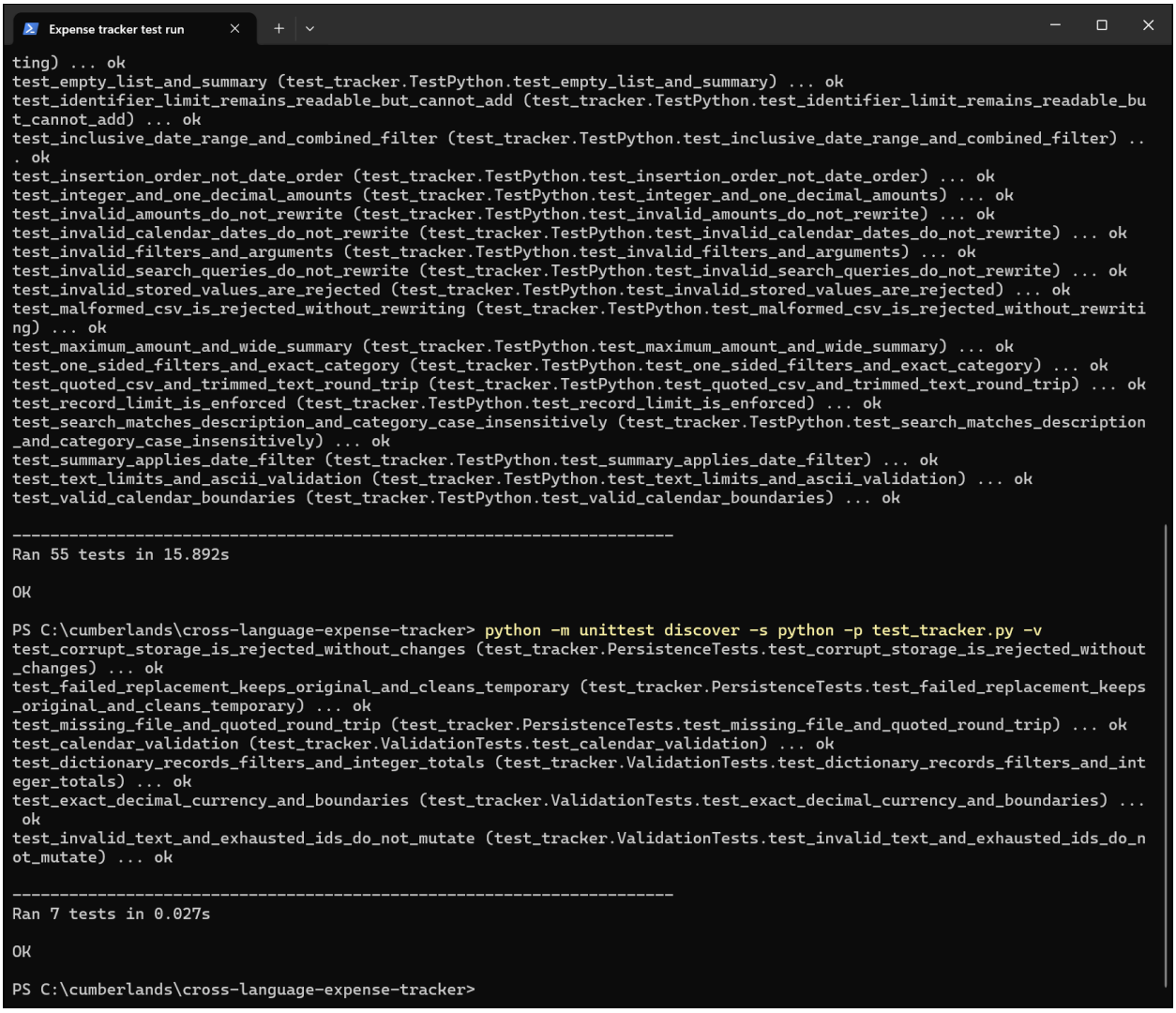
C++ build followed by recording and deleting an expense.



Note. Screenshot of recorded output from command executions. Full transcripts are included in docs/evidence.

Figure 5

Test run output for the behavioral and cross-language suites.



Note. Screenshot of recorded output from command executions. Full transcripts are included in docs/evidence.